

ÖNÁLLÓ LABORATÓRIUM BESZÁMOLÓ

Távközlési és Mesterséges Intelligencia Tanszék

MIKROSZOLGÁLTATÁSOK DINAMIKUS KOMPOZÍCIÓJA AZ EDGE-CLOUD KONTINUUMBAN

Készítette: Morvai Barnabás
Neptun-kód: E0HB0N
Szakirány: Mérnökinformatikus
E-mail cím: morvai.barnabas.bme@gmail.com

Konzulens: Czentye János
E-mail cím: czentye@tmit.bme.hu

Feladat

A félév célja egy olyan keretrendszer megvalósítása, amely serverless felhőalkalmazások atomi függvényeit egy optimalizáló algoritmus kimenetére támaszkodva dinamikusan kompozit Lambda egységekbe szervezi. A hallgató feladata az összeállítási logika implementálása: az orkesztrációs belépési pontok automatikus generálása, a kompozit függvényeken belüli párhuzamos végrehajtás és memória-hatékony állapotkezelés megvalósítása, valamint az AWS infrastruktúra, Lambda függvények, S3 cache és Step Functions állapotgép, Terraform alapú automatizált telepítése.

2025/2026. 2. félév

Tartalomjegyzék

1. Bevezetés	2
1.1. Serverless architektúra, FaaS	2
1.2. AWS ökoszisztéma	2
1.3. Célkitűzés	2
2. Problémafelvetés	2
2.1. Mikroszolgáltatások futási és hálózati költsége	3
2.2. Hívási fa optimalizáció	3
2.3. Optimalizáló	3
3. Rendszerterv és Architektúra	3
3.1. A keretrendszer magas szintű felépítése	4
3.2. Bemeneti adatmodell	4
3.3. Adatfolyam és állapotkezelés	4
3.3.1. Memórián belüli adatátadás	5
3.3.2. Perzisztens adatátadás	5
4. Kompozit függvényen belüli orkesztráció	6
4.1. Standard atomi függvények	6
4.2. Context osztály	6
4.2.1. Belső adatstruktúra	6
4.2.2. Metódusok és működés	6
4.3. Kompozit orkesztrátor dinamikus generálás	7
4.4. Párhuzamosítás	8
5. Felhős orkesztráció	8
5.1. Állapotmentés és betöltés, S3 integráció	8
5.2. Amazon State Language generálás	9
5.3. Terraform infrastruktúra-kiépítés	9
6. Tesztelés és értékelés	10
6.1. Tesztkörnyezet, egy konkrét példa	10
6.2. Költség és teljesítmény összehasonlítás	10
7. Összefoglalás és kitekintés	11
7.1. Eredmények	11
7.2. Továbbfejlesztési lehetőségek	11

1. Bevezetés

Az elmúlt évtizedben a felhőalapú szoftverfejlesztés egyik legmeghatározóbb állomása a serverless architektúra megjelenése volt. Ez a fejezet ennek alapfogalmait, a kapcsolódó AWS ökoszisztémát és a kutatás célkitűzéseit mutatja be.

1.1. Serverless architektúra, FaaS

A hagyományos felhőszolgáltatási modellekkel szemben, ahol a fejlesztőnek gondoskodnia kell a virtuális gépek vagy konténerek üzemeltetéséről és skálázásáról, a serverless modell teljes egészében a felhőszolgáltatóra hárítja az infrastrukturális terheket. A fejlesztő csupán az alkalmazáslogikát implementálja rövid életciklusú, állapotmentes függvények formájában, amelyeket a platform igény szerint indít el és állít le. Ez a Function-as-a-Service modell alapja, ahol a modern platformok automatikus skálázást és pay-as-you-go számlázást biztosítanak. A felhasználó kizárólag a tényleges futási idő és a felhasznált számítási kapacitás alapján fizet.

1.2. AWS ökoszisztéma

A serverless architektúra vezető platformja az Amazon Web Services¹ Lambda szolgáltatása², amely köré egy jól integrált ökoszisztéma épül. A Lambda szorosan együttműködik az AWS többi szolgáltatásával, például S3 eseményekre reagálva triggerelhető, a Step Functions³ pedig komplex munkafolyamatok orkesztrációját teszi lehetővé. Ez az ökoszisztéma rugalmas, natív felhőalkalmazás architektúrákat tesz lehetővé, ahol az egyes Lambda függvények kis, jól definiált feladatokat látnak el.

1.3. Célkitűzés

A dolgozat konzulense által kidolgozott optimalizáló algoritmus meghatározza, hogy egy serverless alkalmazás atomi függvényei hogyan csoportosíthatók költség- és teljesítmény-optimalis kompozit Lambda egységekbe. A jelen laboratóriumi munka célja ennek az optimalizálási eredménynek a gyakorlati megvalósítása. Az optimalizáló JSON kimenetéből dinamikusan generálódnak a kompozit függvények orkesztrációs belépési pontjai és az azokat összefogó AWS Step Functions állapotgép definíció, amelyek alapján a teljes infrastruktúra Terraform⁴ segítségével automatikusan deployolható AWS környezetbe.

2. Problémafelvetés

A serverless alkalmazások költséghatékony üzemeltetése számos, egymással összefüggő tervezési döntést igényel. Ez a fejezet azonosítja azokat a kulcsproblémákat, amelyek a függvénycsoportosítás automatizált optimalizálásának szükségességét indokolják.

¹<https://docs.aws.amazon.com/whitepapers/latest/aws-overview/introduction.html>

²<https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>

³<https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>

⁴<https://developer.hashicorp.com/terraform/intro>

2.1. Mikroszolgáltatások futási és hálózati költsége

A serverless alkalmazások üzemeltetési költsége két összetevőre bontható, a futási költségre és kommunikációs overheadre. A futási költséget a lefoglalt memória és a végrehajtási idő szorzata határozza meg, amelyet a felhőplatformok milliszekundum pontossággal számláznak. Mivel a memória és a CPU-kapacitás arányosan összefügg, a memóriakonfiguráció egyszerre befolyásolja a futási sebességet és a költséget. A hálózati költség a függvények között továbbított adatok méretétől és a közvetítő infrastrukturális elemektől függ. Ha két függvény külső tárolórétegen keresztül ad át köztes eredményt az extra késleltetést és pénzügyi költséget jelent. A függvénycsoportosítás optimalizálásakor ezért mindkét tényezőt együttesen kell kezelni.

2.2. Hívási fa optimalizáció

Serverless környezetben az alkalmazás függvényeit külön-külön Lambda egységekbe szervezni pazarló megközelítés, hiszen minden függvényhívás önálló számlázási egységet képez, beleértve a cold start okozta inicializációs időt és a hálózati kommunikáció overheadjét. Ha több atomi függvényt egyetlen kompozit egységbe csoportosítunk, az adatátadás memórián belül valósulhat meg és a párhuzamos végrehajtás is csökkenti a futási időt. Stabil hívási gráf esetén ezt egy fejlesztő manuálisan is megoldhatja, azonban változó körülmények között, eltérő forgalmi minták, módosuló árak, alkalmazásbővítés stb., más-más csoportosítás lehet az optimális. Az orkesztrációs kód dinamikus generálása erre nyújt megoldást. Ha a csoportosítás eredménye szabványos konfigurációban áll rendelkezésre, az orkesztrátor emberi beavatkozás nélkül előállítható.

2.3. Optimalizáló

Annak meghatározása, hogy az atomi függvények mely kombinációja alkosson kompozit Lambda egységet, NP-teljes optimalizálási probléma⁵, amelynek megoldása a dolgozat konzulense által kidolgozott algoritmus feladata. Az optimalizáló bemenetként megkapja a hívási gráfot és az erőforrás-konfigurációk költség- és teljesítményparamétereit, majd a felhasználó által megadott SLO-követelmények⁶ alapján meghatározza a költségoptimalis particionálást. Az eredményt egy általam definiált JSON konfigurációban adja át, amely tartalmazza a kompozitok nevét és az atomi függvények topologikusan rendezett listáját. A jelen dolgozatban bemutatott keretrendszer ezt a konfigurációt veszi alapul, és az abban leírt csoportosítást valósítja meg az orkesztrátorok generálásától a teljes AWS infrastruktúra kiépítéséig.

3. Rendszerterv és Architektúra

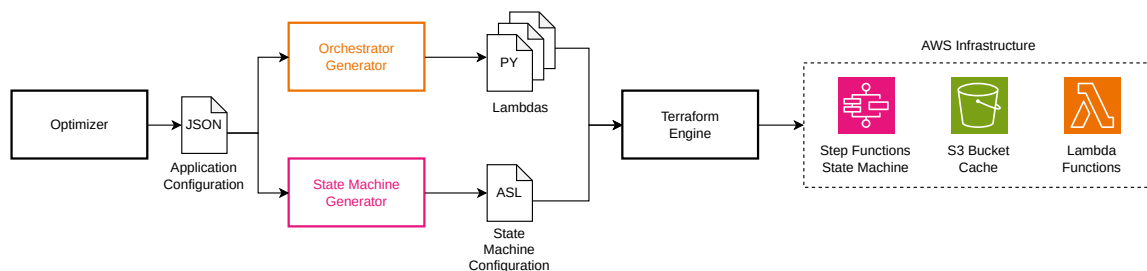
Ez a fejezet a keretrendszer teljes felépítését írja le, az optimalizáló JSON kimenetétől a futásra kész AWS infrastruktúráig. Bemutatja a bemeneti adatmodellt, az adatfolyam kezelésének elveit, valamint azt, hogy az egyes komponensek hogyan illeszkednek egymáshoz a telepítési pipeline során.

⁵<https://www.sciencedirect.com/science/article/pii/S0167739X23004648>

⁶<https://www.ibm.com/think/topics/service-level-objective>

3.1. A keretrendszer magas szintű felépítése

A keretrendszer egy lineáris pipeline mentén épül fel, melynek minden fázisa az előző kimenetére támaszkodik. Az optimalizáló által előállított JSON konfiguráció meghatározza, hogy az atomi függvények milyen csoportosításban alkotnak kompozit Lambda egységeket. Ezt a konfigurációt az orkesztrátor generátor dolgozza fel, minden kompozithoz előállítja a belépési pontot tartalmazó orkesztrációs python fájlt, amely a kompoziton belüli adatáramlást és a párhuzamos végrehajtást vezérli. Az atomi modulok forráskódja és a generált orkesztrátor ezután egyetlen telepíthető csomaggá kerül összeszervezésre. A kész csomagokat és a belőlük generált ASL állapotgép definíciót a Terraform modul tölti fel az AWS infrastruktúrájába, ahol a Lambda függvényeket egy Step Functions állapotgép hangolja össze. A pipeline eredményeként az optimalizáló kimenetéből emberi beavatkozás nélkül áll elő egy teljes mértékben telepített, futásra kész serverless alkalmazás. A keretrendszer telepítési folyamatát a 1. ábra szemlélteti.



1. ábra. A keretrendszer telepítési folyamata: az optimalizáló JSON kimenetétől a futásra kész AWS infrastruktúráig.

3.2. Bemeneti adatmodell

A keretrendszer egyetlen, jól definiált JSON konfigurációs fájlt fogad el bemenetként, amelyet az optimalizáló algoritmus állít elő. A konfiguráció legfelső szintjén a composites kulcs alatt egy lista szerepel, amelynek minden eleme egy kompozit Lambda függvényt ír le. Minden kompozit rendelkezik egy egyedi name mezővel, amely azonosítja a Lambda függvényt és a generált forráskód könyvtárát. Tartalmaz még egy functions listát, amely sorrendben felsorolja a kompozitba tartozó atomi modulokat. Minden atomi bejegyzés két kötelező mezőt tartalmaz. A module mezőt, amely a forráskód könyvtárának nevére mutat, és az input mezőt, amely meghatározza azt a típuskulcsot, amelyen keresztül az adott modul a context példányból a bemenetét várja. Ez a minimális, lapos struktúra szándékosan kerül a fa topológia explicit leírását, az adatáramlás irányát nem a konfiguráció, hanem a kontextus mechanizmus határozza meg futásidőben.

3.3. Adatfolyam és állapotkezelés

A rendszer adatkezelési stratégiája két egymást kiegészítő rétegre épül, amelyek elválasztják egymástól a kompozit függvényen belüli, rövid életű köztes adatokat és a kompozit határokat átlépő, perzisztens állapotokat. Az adatfolyam helyességének elméleti alapját a topologikus rendezés adja. Mivel minden hívási gráf egy irányított körmentes gráf és minden DAG rendelkezik topologikus rendezéssel, az atomi függvények végrehajtási sorrendje meghatározható úgy, hogy minden függvény csak azután kerüljön sorra, miután

az összes tőle függő függvény már lefutott. Ez a tulajdonság közvetlenül következik abból, hogy minden fa egyben DAG is. A JSON konfigurációban ezért elegendő az atomi függvényeket topologikus sorrendben felsorolni és megadni a bemeneti típuskulcsukat, az explicit fa struktúra leírása felesleges. Amikor a végrehajtás egy adott atomi függvény szinkronizációs pontjához ér, a szükséges bemeneti adatok a context példányban már biztosan rendelkezésre állnak, hiszen ha nem így lenne, az azt jelentené, hogy egy olyan függvénytől vár inputot, amely a sorrendben csak utána következik, ez pedig visszaélő nyilat feltételezne a gráfban, ami körmentes gráfban definíció szerint lehetetlen. Ugyanez a logika a magasabb szintű, kompozit függvények közötti adatáramlásra is érvényes, abban a dimenzióban a közös memória szerepét egy központi S3 bucket⁷ tölti be.

3.3.1. Memórián belüli adatátadás

Egy kompozit Lambda függvény végrehajtásának teljes időtartama alatt az atomi függvények közötti adatátadás kizárólag memórián belül, a megosztott context példányon keresztül történik. Ez a megközelítés kiküszöböli a hálózati hívások okozta késedelmet és a cache szolgáltatás igénybevételeből eredő járulékos költségeket, amelyek az egymást láncolva meghívó önálló Lambda függvények esetén jelentős overheadet jelentenének. A memóriahatékonyságot a destruktív olvasás biztosítja. A context.get hívás minden esetben eltávolítja az átadott adatot a tárolóból, így a már feldolgozott köztes eredmények azonnal felszabadulnak, és nem terhelik a Lambda korlátozott memória erőforrását a végrehajtás hátralévő részében.

3.3.2. Perzisztens adatátadás

Amikor egy kompozit függvény, tehát egy Lambda függvény eléri az utolsó atomi függvényt, a context példányban visszamaradó rekordok kizárólag olyan kimenetek lehetnek, amelyeket egy másik kompozit függvény fog felhasználni bemenetként. Ez a destruktív olvasás közvetlen következménye. Minden adat, amelyet az adott kompozit valamely atomi függvénye már feldolgozott, lekérdezéskor törlődött az adatstruktúrából, így csakis a külső függőségek számára fenntartott kimenetek maradnak. Ezek az adatok a fogadó kompozitban biztosan egy gyökérfüggvényhez érkeznek, ha ez nem így lenne, a hívási gráf nem fa, hanem általánosabb DAG lenne. A végrehajtás végén a kompozit függvény a kontextusban visszamaradt kimeneteket egy központi S3 bucketbe menti. A fogadó kompozit Lambda függvény indításakor a beolvasó függvény visszatölti az előkészített adatot, és azt közvetlenül a saját context példányába regisztrálja. Ez a kétszintű modell lehetővé teszi, hogy a kompozit határokat az AWS Step Functions State Machine vezérelje anélkül, hogy nagy mennyiségű adatot kellene az ASL állapotátmenetek payloadjában átvinni, mivel az érdemi tartalom az S3 bucketen keresztül áramlik, az állapotgép üzenetei pedig csupán az aktuális futás és a feldolgozandó adat azonosítóját hordozzák.

Mivel a keretrendszernek tetszőleges Python objektumokat kell tudnia átvinni kompozit határon a típusrendszer semmilyen megkötést nem támaszt az atomi függvények kimeneteire. Az S3-ba való kiírás előtt az objektumok szerializációja szükséges. Erre a Python pickle⁸ modulja nyújt általános megoldást. A kiíró függvény minden kimeneti objektumot pickle formátumba sorosít, majd bináris objektumként tölti fel az S3 bucketbe. A beolvasó függvény ennek fordítottját végzi, a bináris tartalmat visszaalakítja

⁷<https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>

⁸<https://docs.python.org/3/library/pickle.html>

Python objektummá, és azt közvetlenül regisztrálja a fogadó kompozit context példányába, ahonnan az első atomi függvény típuskulcs alapján érheti el. Az atomi függvények implementációja szempontjából nincs különbség, hogy egy adat memórián belülről vagy az S3 cache-ből érkezett.

4. Kompozit függvényen belüli orkesztráció

A serverless alkalmazások optimalizálásának eredményeként előálló kompozit Lambda függvények belső működéséhez egy egységes, automatizáltan generálható orkesztrációs mechanizmusra van szükség. Ez a fejezet azt írja le, hogy az optimalizáló által csoportosított atomi függvények hogyan kerülnek egyetlen telepíthető egységbe összeszervezésre, és hogy a közöttük lévő adatáramlás hogyan valósul meg futásidőben.

4.1. Standard atomi függvények

Az atomi függvények a rendszer legkisebb, önállóan is futtatható feldolgozási egységei. Ahhoz, hogy ezeket az egységeket az orkesztrátor dinamikusan, konfigurációvezérelt módon hívhassa meg, minden atomi modul belépési pontjának egységes, előre rögzített interfésszel kell rendelkeznie. Az interfész két paramétert ír elő. Az első egy Any típusú Python objektum, amely az adott modul által feldolgozandó bemeneti adatot hordozza. A második egy context osztály példánya, amelybe a modul a feldolgozás eredményeit regisztrálja. Ez a standardizált interfész lehetővé teszi, hogy az orkesztrátor generátor ne függjön az egyes modulok konkrét implementációjától, elegendő ismerni a modul nevét és a várt bemenet típusának kulcsát.

4.2. Context osztály

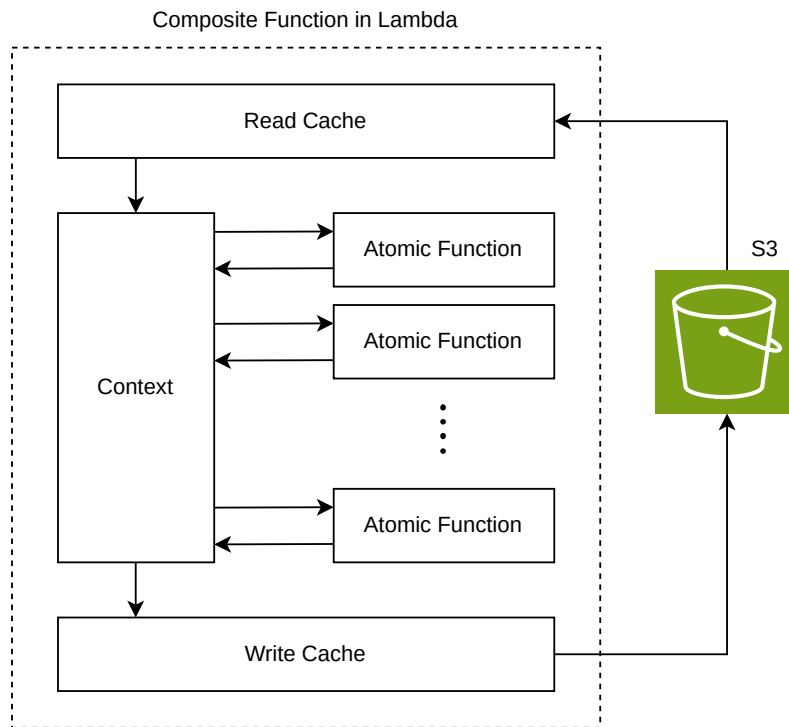
A context osztály a kompozit függvényen belüli adatáramlás központi eleme. Feladata, hogy az atomi függvények között közvetítse a köztes eredményeket anélkül, hogy azok külső cache szolgáltatáshoz kellene fordulniuk. Minden kompozit Lambda hívás elején egy új context példány jön létre, amely egy egyedi futási azonosítóhoz van kötve. Ez biztosítja, hogy párhuzamos hívások esetén az egyes futások adatai ne keveredjenek össze.

4.2.1. Belső adatstruktúra

A context osztály egy `defaultdict(list)` típusú adatstruktúrát használ, amelynek kulcsai string típusúak, értékei pedig az adott típushoz tartozó objektumok listái. Ez a struktúra természetes módon kezeli azt az esetet, amikor egy atomi függvény egy adott típusból több kimenetet is előállít, mivel minden register hívás egyszerűen hozzáfűzi az új elemet a megfelelő kulcshoz tartozó listához. A kompozit függvényen belüli adatáramlást és az S3 alapú állapotkezelést a 2. ábra szemlélteti.

4.2.2. Metódusok és működés

A context osztály három elsődleges műveletet tesz elérhetővé. Az objektumokat regisztráló metódus segítségével egy atomi függvény a kimenetét egy string típuskulcs alá regisztrálhatja az adatstruktúrában. A kulcs szerinti lekérdező metódus a következő atomi függvény számára elérhetővé teszi az adott kulcshoz tartozó összes objektumot. Ezt destruktív olvasással történik, a `defaultdict` belső `pop` metódusát alkalmazva visszaadja a



2. ábra. Adatáramlás és orkesztráció a kompozit függvényen belül a belső Context struktúra és a külső S3 állapotkezelő segítségével.

kulcshoz tartozó lista referenciáját, majd azonnal törli azt a tárolóból. Ez a megközelítés memóriahatékonysági szempontból kritikus, mivel a Lambda függvények esetében a rendelkezésre álló memória korlátozott és közvetlen költségfaktor. A már feldolgozott adatok azonnali felszabadítása jelentős költségcsökkenést eredményezhet, különösen akkor, ha a kompozit függvény sok atomi függvényből áll, vagy az egyes lépések nagy mennyiségű köztes kimenetet generálnak. Az osztály emellett iterálható, a megfelelő metódusokon keresztül az orkesztrátor lekérdezheti, hogy az adatstruktúrában éppen milyen típuskulcsok szerepelnek, ami a párhuzamos futtatás logikai alapja.

4.3. Kompozit orkesztrátor dinamikus generálás

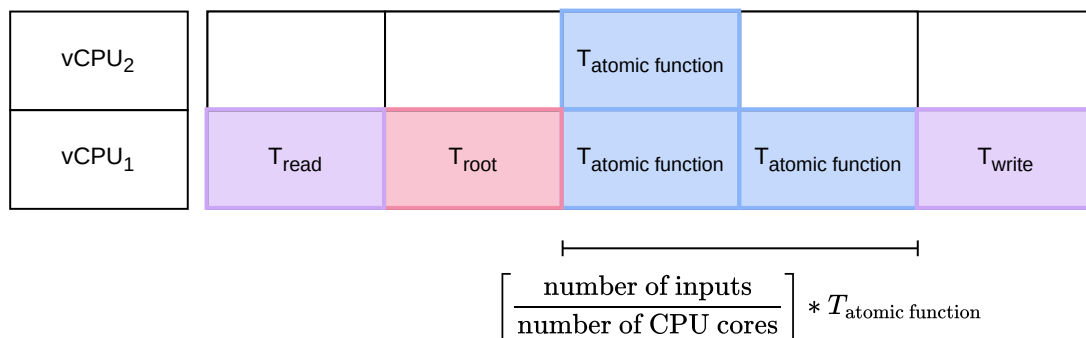
A sablon a kompozit functions listájából kiszámítja az összes szükséges import utasítást, meghatározza az első atomi függvény nevét és bemenet-kulcsát, majd felsorolja a párhuzamosan indítható további lépéseket. Az így előálló függvény az AWS Lambda belépési pontjaként közvetlenül hívható, és tartalmazza a futási azonosító kezelését, a cache olvasás/írás logikáját, valamint a párhuzamos hívásokat. A generátor megkülönbözteti a gyökér kompozitot a párhuzamosan futtatható kompozitoktól. A gyökér kompozit maga állítja elő az egyedi futási azonosítót, míg a többi kompozit azt az esemény payload-jából olvassa ki, és emellett a beolvasó függvény segítségével előkészíti a feldolgozandó objektumot a context példányba.

A közös context példány bevezetése a konfigurációs JSON-séma jelentős egyszerűsítését is lehetővé tette. A korábbi megközelítésben minden atomi függvény egy Dict objektumba

helyezte kimeneteit, és az egymás utáni függvények közötti adatkapcsolatot explicit fa struktúraként kellett a JSON-ban is leképezni, ez a konfigurációt átláthatatlanná tette és az adatok kinyeréséhez regex alapú mintaillesztésre volt szükség, amelyhez pontosan ismerni kellett, hogy melyik kimenet melyik függvénytől származik. A közös context használatával ez a fa topológia eltűnik a konfigurációból. Elegendő csupán a kompozithoz tartozó atomi függvények lineáris listáját és azok típuskulcsait megadni, az adatáramlás irányát pedig a register és get hívások párosítása határozza meg futásidőben. Feltételezve, hogy az optimalizáló, a hívási fa topologikus sorrendjében határozza meg a függvények sorrendjét.

4.4. Párhuzamosítás

A kompozit függvényen belüli párhuzamosítást a parallel segédfüggvény valósítja meg, amely egy adott típuskulcs alatt összegyűlt összes elemet párhuzamosan, külön szálban dolgozza fel. A függvény először destruktív olvasással lekérdezi a context példányból a feldolgozandó elemek listáját. A párhuzamos szálak nem írhatnak közvetlenül a megosztott context példányba, mivel nem szálbiztos. Ennek megoldására egy proxy közvetítő osztály egy megosztott listára mutat, amelybe a szálak zárolás nélkül rögzíthetik az eredményeiket. A párhuzamos végrehajtás befejezése után az összegyűlt eredmények visszaregisztálásra kerülnek az eredeti context példányba. Ezt a végrehajtási szekvenciát szemlélteti a 3. ábra, amelyen jól követhető, ahogy az input beolvasása (T_{read}) és az első atomi függvény (T_{root}) után a további atomi függvények ($T_{atomicfunction}$) párhuzamosan hajtódnak végre, a folyamatot pedig a kimenetek kiírása (T_{write}) zárja.



3. ábra. A kompozit függvényen belüli párhuzamos végrehajtás időzítési modellje és a feladatok elosztása a virtuális processzormagok között.

5. Felhős orkesztráció

Ez a fejezet a kompozit Lambda függvények közötti, magasabb szintű koordinációt írja le. Az S3 alapú állapotkezelést, a Step Functions állapotgép automatikus generálását, valamint a teljes infrastruktúra Terraform segítségével történő kiépítését.

5.1. Állapotmentés és betöltés, S3 integráció

A kompozit Lambda függvények közötti vezérlési folyamat kezelésére az AWS Step Functions szolgáltatás nyújt megfelelő megoldást. Bár technikailag lehetséges Lambda függ-

vényekből más Lambda függvényeket meghívni, ez a megközelítés nem ajánlott, a hívó függvény a teljes végrehajtási idő alatt fut és fizet, miközben csupán a hívott függvény befejezésére vár. A Step Functions ezzel szemben natív módon kezeli az állapotátmeneteket és a párhuzamos végrehajtást, a Lambda függvények futási ideje alatt nem tart fenn erőforrásokat, és az állapotok közötti adatkoordinációt a keretrendszer végzi.

A kompozit függvények közötti adatátadás egy központi S3 bucket közvetítésével történik, ami cache szolgáltatásként funkcionál. Amikor egy kompozit Lambda függvény befejezi a végrehajtását, a kiíró függvény a context példányban visszamaradt kimeneteket típuskulcs szerinti könyvtárstruktúrába rendezve tölti fel az S3 bucketbe, a stepfunctions-cache/<futási azonosító>/<típuskulcs>/ elérési út alá. Minden kimenet önálló objektumként kerül tárolásra, így az S3 könyvtár elemszáma megfelel az adott típusból generált kimenetek számának. Ez teszi lehetővé, hogy a Step Functions a következő állapotban az S3 könyvtár listázásával meghatározza, hány párhuzamos Lambda példányt kell indítani, és az egyes példányoknak melyik objektum kulcsát kell átadni.

5.2. Amazon State Language generálás

Az állapotgép definíciója Amazon States Language formátumban készül, amelyet a keretrendszer az optimalizáló JSON kimenetéből automatikusan generál Jinja2 sablonnal. A generátor bemenetként megkapja a kompozitok listáját, az S3 bucket nevét, az AWS azonosítókat, amelyekből felépíti a Lambda függvények és S3 ARN hivatkozásait.

A generált állapotgép szerkezete közvetlenül tükrözi a hívási fa topológiáját. A kompozitok listájának első eleme mindig a gyökér kompozit, amely egyetlen Task állapotként kerül az állapotgépbe. Ez a függvény a teljes futás során pontosan egyszer hívódik meg, létrehozza a futási azonosítót és az S3 bucketbe írja az első köztes eredményeket. A lista összes további eleme Distributed Map állapotként jelenik meg, amely az S3 könyvtár tartalmát listázva dinamikusan meghatározza a párhuzamosan indítandó Lambda példányok számát. Minden Map állapot egy beágyazott Task állapotot tartalmaz, amely az egyes S3 objektumok kulcsát és a futási azonosító értékét adja át a megfelelő kompozit Lambda függvénynek. Az állapotok között az átmenetet a Next mező biztosítja, az utolsó állapot pedig End jelzéssel zárja le a végrehajtást.

5.3. Terraform infrastruktúra-kiépítés

A generált Lambda csomagok és az ASL állapotgép definíció telepítése Terraform segítségével, Infrastructure as Code-ként valósul meg. Ez biztosítja, hogy az optimalizáló által meghatározott konfiguráció minden változása reprodukálható és verziókezelten alkalmazható legyen az AWS környezetben.

A Terraform modul először egy globálisan egyedi S3 bucketet hoz létre, amely a kompozit függvények közötti cache szolgáltatásként funkcionál, és amelynek neve az összes Lambda függvény környezeti változóján keresztül kerül átadásra. Ezt követően a modul dinamikusan felderíti a generált kompozit könyvtárakat, és minden talált alkönyvtárból önálló Lambda függvényt hoz létre, beégetett névlista nélkül. Ez azt jelenti, hogy az optimalizáló kimenetének változásakor, akár több, akár kevesebb kompozit keletkezik, a Terraform konfiguráció módosítás nélkül alkalmazható újra.

A megosztott keretrendszer kód, egy külön Lambda réteggként kerül telepítésre. Ebbe tartozik a kompozit függvényen belüli átadásért felelős context osztály, a párhuzamosításért, ki- és beolvasásért felelős függvények. A layer buildje során a Terraform lokálisan

lefuttatja a pip telepítést, majd az eredményt zip archívumba csomagolja és feltölti az AWS-be. Minden dinamikusan generált Lambda függvény ezt a layert használja, így a keretrendszer kódja egyetlen helyen verziókezel, és nem kerül be minden egyes Lambda deployment csomagba külön-külön. A szükséges IAM jogosultságokat egy előre kiosztott szerepkör biztosítja, amely az AWS Academy környezet standard megoldása, éles környezetben természetesen érdemes lecserélni.

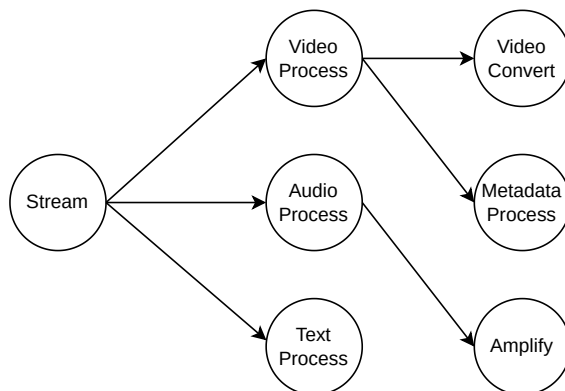
Végül a modul az ASL fájlból létrehozza a Step Functions állapotgépet, amelynek regisztrálása csak az összes Lambda függvény sikeres feltöltése után történik meg.

6. Tesztelés és értékelés

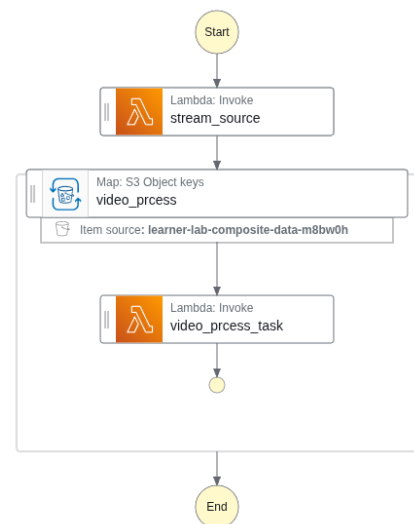
A keretrendszer működésének szemléltetésére egy valós AWS környezetben futtatott teszttel kerül bemutatásra, amely egy egyszerű médiafeldolgozó alkalmazást modellez.

6.1. Tesztkörnyezet, egy konkrét példa

A teszthez használt alkalmazás hívási fája hat atomi függvényből áll: a gyöker Stream függvény Video Process és Audio Process ágakra bomlik, amelyek további Video Convert, illetve Text Process és Amplify függvényeket hívnak, ahogy azt a 4. ábra mutatja. Az atomi függvények dummy implementációk, amelyek egyszerű string értékeket adnak át egymásnak, így a mért idők kizárólag az infrastrukturális overheadet tükrözik, nem a tényleges feldolgozási logikát. A teszt két konfigurációban futott le. Az elsőben minden atomi függvény önálló Lambda egységként került telepítésre, a másodikban az optimalizáló két kompozitba szervezte őket, a Video Process részfája egy egységet alkotott, a Stream függvény pedig egy másikat. A két kompozitra bontott konfiguráció automatikusan generált Step Functions állapotgépét a 5. ábra szemlélteti.



4. ábra. A tesztelés hívási fája.



5. ábra. Generált állapotgép.

6.2. Költség és teljesítmény összehasonlítás

A mérési eredmények jelentős különbséget mutatnak a két konfiguráció között. A hat önálló Lambda esetén a teljes végrehajtási idő 16,07 másodperc volt, míg a két kompozitra

bontott verzió ugyanezt 2,93 másodperc alatt teljesítette, ami közel hatszoros gyorsulásnak felel meg. A tesztet többször lefuttatva a cold start hatása kizárható, a különbség tehát kizárólag a Lambda függvények közötti hívások overheadjéből ered: minden egyes kompozit határon az S3-ba való kiírás és visszaolvasás, valamint a Step Functions állapotátmenet járulékos késleltetése összeadódik. Hat önálló Lambda esetén ez hat alkalommal merül fel, míg két kompozit esetén csak egyszer, a két egység határára. Mivel a pay-as-you-go elszámolás a futási idő alapján történik, a végrehajtási idő csökkenése közvetlen költségcsökkenést is jelent. Fontos megjegyezni, hogy a tesztben használt dummy implementációk minimális tényleges feldolgozási munkát végeznek, így a mért idő szinte teljes egészében infrastrukturális overhead. Éles, számításigényes alkalmazásban az arány a tényleges feldolgozási idő javára tolódna el, azonban az átadásokból eredő abszolút késleltetés azonos maradna, és nagy számú kompozit esetén továbbra is meghatározó tényező lenne a teljes végrehajtási időben.

7. Összefoglalás és kitekintés

A fejezet összefoglalja a laboratóriumi munka során elért eredményeket, és azonosítja azokat a területeket, amelyeken a keretrendszer tovább fejleszthető.

7.1. Eredmények

A laboratóriumi munka eredményeként elkészült egy olyan keretrendszer, amely az optimalizáló JSON kimenetéből kiindulva, emberi beavatkozás nélkül képes előállítani és telepíteni egy teljes serverless alkalmazást AWS környezetben. A keretrendszer automatikusan generálja a kompozit Lambda függvények orkesztrációs belépési pontjait, a köztük lévő adatáramlást koordináló Step Functions állapotgép definícióját, valamint a teljes infrastruktúrát leíró Terraform konfigurációt. A teszteredmények igazolták, hogy a kompozit összevonás érdemi teljesítményjavulást és ezzel együtt költségcsökkenést eredményez: a hat önálló Lambda helyett két kompozitot alkalmazó konfiguráció közel hatszoros gyorsulást eredményezett a bemutatott teszt esetében azonos terhelés mellett. A keretrendszer moduláris felépítése lehetővé teszi, hogy az optimalizáló kimenetének változásakor az újratelepítés egyetlen pipeline futtatással elvégezhető legyen.

7.2. Továbbfejlesztési lehetőségek

A jelenlegi implementáció legnagyobb teljesítménybeli korlátja a kompozit függvények közötti adatátadásra használt S3 bucket, amelynek egy írási vagy olvasási művelete jellemzően 50–250 ms késleltetéssel jár. Ennek kiküszöbölésére in-memory cache szolgáltatás alkalmazható, például AWS ElastiCache for Redis⁹, amely ezredmásodperc alatti sebességet garantál, cserébe magasabb üzemeltetési költség mellett. A cache réteg cseréje minimális módosítást igényelne, mivel a ki- és beolvasó függvények implementációja lecserélhető anélkül, hogy az atomi függvények vagy az orkesztrátor generátor érintett lenne.

További fejlesztési irány a Terraform konfiguráció kiterjesztése, amely jelenleg minden Lambda függvényt azonos mennyiségű memóriával telepít. Az optimalizáló kimenetét memóriakonfigurációval kiegészítve függvényenként eltérő erőforrás-kiosztás válna lehetővé.

⁹<https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/WhatIs.html>